

DÉCADA

O que falta fazer · plano de trabalho do grupo · 22 de setembro de 2026

Este documento existe para que qualquer pessoa do grupo consiga pegar uma tarefa sem precisar perguntar por onde começar. Cada tarefa diz o que é, por que importa, em quais arquivos se mexe e como saber que terminou.

Onde o projeto está hoje

O backend está completo para o MVP e o aplicativo percorre a jornada inteira: entrar, enviar a prescrição em PDF, confirmar os produtos, ver a lista de compras com custo, cadastrar a despensa e ver as receitas e a economia.

Camada	Situação	Testes
Backend (FastAPI, SQLAlchemy, Alembic)	Completo para o MVP: 17 rotas, 6 migrações, todas as regras de negócio com teste	96 passando
Aplicativo (React Native + Expo)	Seis telas e quatro abas: Dieta, Mercado, Despensa e Economia	85 passando
Banco (PostgreSQL 16)	Sobe por Docker Compose; seed de desenvolvimento com 120 produtos, 1080 preços e 10 receitas, todos marcados como fictícios	—
Documentação	Modelo de dados, LGPD, protótipos, sistema de design, contrato das telas, dossiê técnico e roteiro de demonstração	—

Como montar o ambiente

Na primeira vez, na raiz do repositório:

```
docker compose up -d
cd backend
python3 -m venv .env
.venv/bin/pip install -e ".[dev]"
.venv/bin/alembic upgrade head
.venv/bin/python -m app.seeds
cd ../mobile && npm install
```

No dia a dia, dois terminais:

```
cd backend && .venv/bin/uvicorn app.main:app --reload
cd mobile && npx expo start
```

Antes de abrir qualquer alteração, os dois conjuntos de teste precisam passar:

```
cd backend && .venv/bin/pytest -q
cd mobile && npm test
```

O seed imprime a conta de desenvolvimento para entrar no aplicativo. Todo dado do seed é fictício e está marcado como tal — nenhum preço foi coletado de mercado real.

As tarefas, em ordem de valor

A ordem não é obrigatória, mas a tarefa 1 destrava as outras duas da aba Despesa e por isso vem primeiro.

1. Vincular e medir um item já salvo

backend + app

O que é. Hoje só dá para escolher o produto e informar a quantidade no momento de adicionar o item. Quem salvou "aveia" sem produto, ou sem quantidade, não tem como corrigir depois: só apagando e cadastrando de novo.

Por que importa. Item sem produto não conta para nada; item com produto e sem quantidade conta para as receitas mas não abate da compra. A tela já avisa das duas situações — falta dar o conserto.

Onde mexer. Criar PATCH `/pantry/{item_id}` em `backend/app/api/routes/pantry.py`, aceitando `product_id`, `quantity` e `unit`. Não há mudança de modelo nem migração: as colunas já existem. No aplicativo, abrir o mesmo passo de medida ao tocar em um item da despesa, em `mobile/src/screens/PantryScreen.tsx`.

Pronto quando. Um item salvo só como texto vira item vinculado e medido sem ser apagado; o aviso correspondente some da tela; há teste de rota no backend e teste de tela no aplicativo.

2. Região escolhida por quem usa

app

O que é. A lista de compras é gerada com 'DF', 'Brasília' escritos no código, em `mobile/src/screens/MarketScreen.tsx`, linha 52.

Por que importa. A decisão 2 do projeto diz que a média de preço é sempre por região, nunca nacional. Uma região fixa no código faz o número certo aparecer para a cidade errada.

Onde mexer. No aplicativo: um seletor de estado e cidade antes de gerar a lista, guardando a escolha na sessão. O backend já recebe a região no corpo da requisição e não precisa mudar — a decisão de grupo é se a região passa a ser um campo do usuário no banco (aí há migração) ou fica só na sessão do app.

Pronto quando. A pessoa escolhe a região, a lista é gerada para ela, e a tela continua mostrando de qual cidade veio o preço.

3. Item avulso na lista de compras

backend + app

O que é. Só entra na lista o que veio do plano alimentar. Não há como acrescentar o sabão em pó que também precisa ser comprado.

Por que importa. É a diferença entre uma lista que se usa no mercado e uma lista que se consulta e se abandona.

Onde mexer. Tornar `plan_item_id` nulo em `ShoppingListItem` (`backend/app/models/shopping_list.py`) — isso exige migração Alembic e revisar a restrição de unicidade `uq_shopping_list_items_lista_item`, que hoje pressupõe o campo preenchido.

Cuidado. Item avulso não é substituição de alimento prescrito. A decisão 8 do projeto proíbe o aplicativo trocar o que a nutricionista pediu; acrescentar item que não é comida da dieta é outra coisa.

Pronto quando. Dá para acrescentar e remover item avulso, ele entra no custo estimado, e o teste garante que item do plano continua único na lista.

O que é. Ler o QR Code da nota fiscal do supermercado e transformar a compra em registros de preço reais.

Por que está aqui. A decisão 3 do projeto chama a NFC-e de fonte **principal** de preço. Hoje nfce existe apenas como valor de enumeração: não há leitor nem interpretador. Todo preço do ambiente vem do seed e é fictício.

Tamanho. Grande: leitor de QR Code no aplicativo, requisição ao portal da SEFAZ, interpretação do HTML de retorno, casamento de cada item da nota com o catálogo, e teste para cada uma dessas partes. É a única tarefa desta lista que não cabe numa sessão de trabalho.

Recomendação. Só começar depois que 1, 2 e 3 estiverem fechadas. Se o prazo apertar, ela fica registrada como trabalho futuro na apresentação — o que já está documentado como decisão consciente, não como esquecimento.

Regras de trabalho

Estas valem para todo mundo e estão no CLAUDE.md do repositório:

- Combine o plano antes de escrever código. Uma etapa por vez.
- Commits pequenos e descritivos, em português, no padrão Conventional Commits (feat:, fix:, refactor:, test:, docs:).
- Todo módulo de regra de negócio precisa de teste. Sem exceção para casamento item–produto, cálculo de preço e interpretação de NFC-e.
- Nunca inventar dado de preço. Dado fictício só no seed, e marcado como tal.
- Código e comentários em português; nomes de variáveis, funções e tabelas em inglês.
- Decisão de arquitetura ambígua se discute no grupo antes, não se escolhe sozinho.

Decisões que não se reabrem sem discutir

São dez, e estão inteiras no CLAUDE.md. As quatro que mais afetam as tarefas acima:

- **Preço nunca é um valor único:** média, mediana e desvio, com data e origem. Preço com mais de 30 dias aparece como estimativa.
- **A média é por região,** nunca nacional.
- **O aplicativo não prescreve dieta.** Ele operacionaliza uma prescrição que já existe. Substituição de alimento só existe se veio da nutricionista.
- **Plano alimentar é dado sensível de saúde.** Consentimento explícito, minimização e exclusão sob demanda são requisitos, não melhorias.

O que ficou de fora do MVP

Registrado para ninguém gastar tempo achando que é esquecimento:

Fora do MVP	Motivo
Leitura de foto de papel (OCR)	Import de plano é PDF com texto
Refeições com horário e status, hidratação, carrossel semanal	O aplicativo não gerencia a rotina alimentar, só a compra
Projeção mensal e desperdício evitado	Exigiria saber o que foi consumido antes de vencer, e o item da despensa não guarda validade

Fora do MVP	Motivo
Dicas de substituição geradas pelo aplicativo	Contraria a decisão 8: só a nutricionista substitui alimento
Exportar para WhatsApp e fotos de receita	Não sustentam o MVP

Onde ler mais

- CLAUDE.md — contexto, stack, domínio e as dez decisões.
- README.md — instalação, migrações e o que falta.
- docs/mode lo-dados.md — diagrama e a razão de cada regra de exclusão.
- docs/lgpd.md — consentimento, minimização e exclusão, cada um com o teste que o sustenta.
- docs/design/te las.md — o que cada tela exige do backend, e as divergências entre protótipo e decisões.
- docs/demo/roteiro.md — roteiro da demonstração.